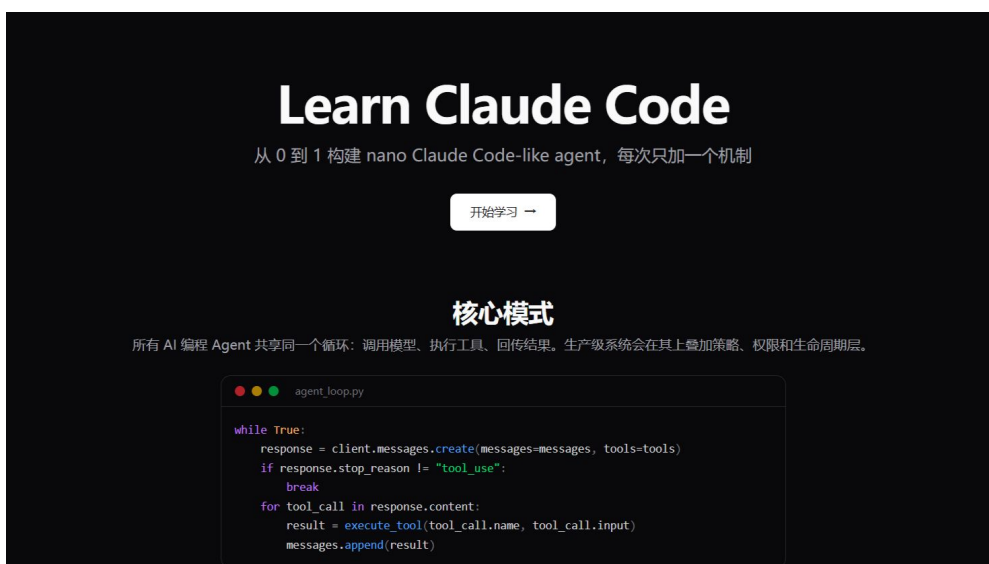


课程笔记

【Learn Claude Code 源码详解】 - 上下文压缩，换来无限会话

五道口纳什 & Codex

2026 年 4 月 15 日



视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 24:38

视频链接: <https://www.bilibili.com/video/BV1xBw5z6E4F/>

目录

1 三层上下文压缩与 micro_compact 初探	3
1.1 为什么需要上下文压缩	3
1.2 调试环境搭建	3
1.3 第一次进入 micro_compact	4
1.4 第二轮与第三轮：开始收集 tool_results	5
1.5 首次触发替换：内容被压缩为占位符	6
1.6 本章小结	7
2 micro_compact 的首次执行与工具结果压缩机制	7
2.1 为什么要先进行微小压缩？	7
2.2 tool_results 的收集与观察	8
2.3 构建 name_map：工具 ID 到工具名称的映射	8
2.4 替换规则：长度大于 100 即压缩	9
2.5 第一次压缩的实际效果	10
2.6 本章小结	10
3 上下文压缩：auto_compact 的触发与入口条件	11
3.1 动机：为什么需要上下文压缩？	11
3.2 auto_compact 的入口与触发条件	11
3.3 Token 估算：estimate_tokens 的逻辑	12
3.4 50,000 Token 阈值：压缩的临界点	13
3.5 模型主导的 compact：让 LLM 自己决定何时压缩	13
3.6 本章小结	13
4 Auto Compact：磁盘持久化与 LLM 摘要压缩	14
4.1 动机：为什么要做“自动压缩”	14
4.2 创建 .transcripts 目录并保存原始消息	14
4.3 LLM 摘要调用：summarize this conversation for continuity	15
4.4 80,000 字符截断：被丢弃的尾部信息	15
4.5 从模型响应到压缩后的消息列表	16
4.6 本章小结	16
5 调试中的意外发现：三层上下文压缩与潜藏的 Bug	17
5.1 从 micro_compact 开始：旧工具结果的「微瘦身」	17
5.2 auto_compact：客户端的 5 万 Token 警戒线	17
5.3 模型主导的第三层压缩：手动触发 compact	18
5.4 源码中的 Bug：thinking block 没有 text 属性	18
5.5 80k 字符截断：压缩带来的隐性风险	19
5.6 本章小结	19

6 总结与延伸	20
6.1 讲师的收官之言	20
6.2 核心论断的结构化提炼	20
6.3 三层机制的交叉关联	20
6.4 工程实践的启示	21
6.5 开放问题与未来方向	21
6.6 最后的延伸思考	21

1 三层上下文压缩与 `micro_compact` 初探

1.1 为什么需要上下文压缩

大语言模型 (LLM) 的上下文窗口并非无限。当对话轮次增多、工具调用结果累积时，输入给模型的消息列表会越来越长，最终可能超出模型能处理的 token 上限。Claude Code 的源码通过一套分层的压缩策略来解决这个问题：

1. **微观压缩** (`micro_compact`)：在每次循环中自动替换旧的工具结果为占位符。
2. **客户端自动压缩** (`auto_compact`)：当估算 token 超过阈值（5 万）时，客户端主动把所有消息摘要成两条。
3. **模型驱动的手动压缩** (`compact` 工具)：让 LLM 自己决定何时调用压缩工具。

本节聚焦于源码调试环境的搭建，以及第一层压缩——`micro_compact` 的初次观察。

核心目标

通过 VS Code 调试器逐步跟踪 `micro_compact` 的执行过程，理解它是如何在消息列表中收集旧工具结果、并用短占位符替换长内容的。

1.2 调试环境搭建

打开 VS Code 后，定位到 `S06_context_compact` 示例。为了跟踪压缩逻辑，需要在以下关键位置设置断点：

- `micro_compact` 函数入口（约第 69 行）
- `auto_compact` 函数入口（约第 99 行）
- Agent 主循环中调用压缩逻辑的分支（约第 228 行）

与之前的示例相比，主函数结构没有变化，但 Agent Loop (约第 197 行) 新增了先调用 `micro_compact` 再进入后续处理的逻辑。

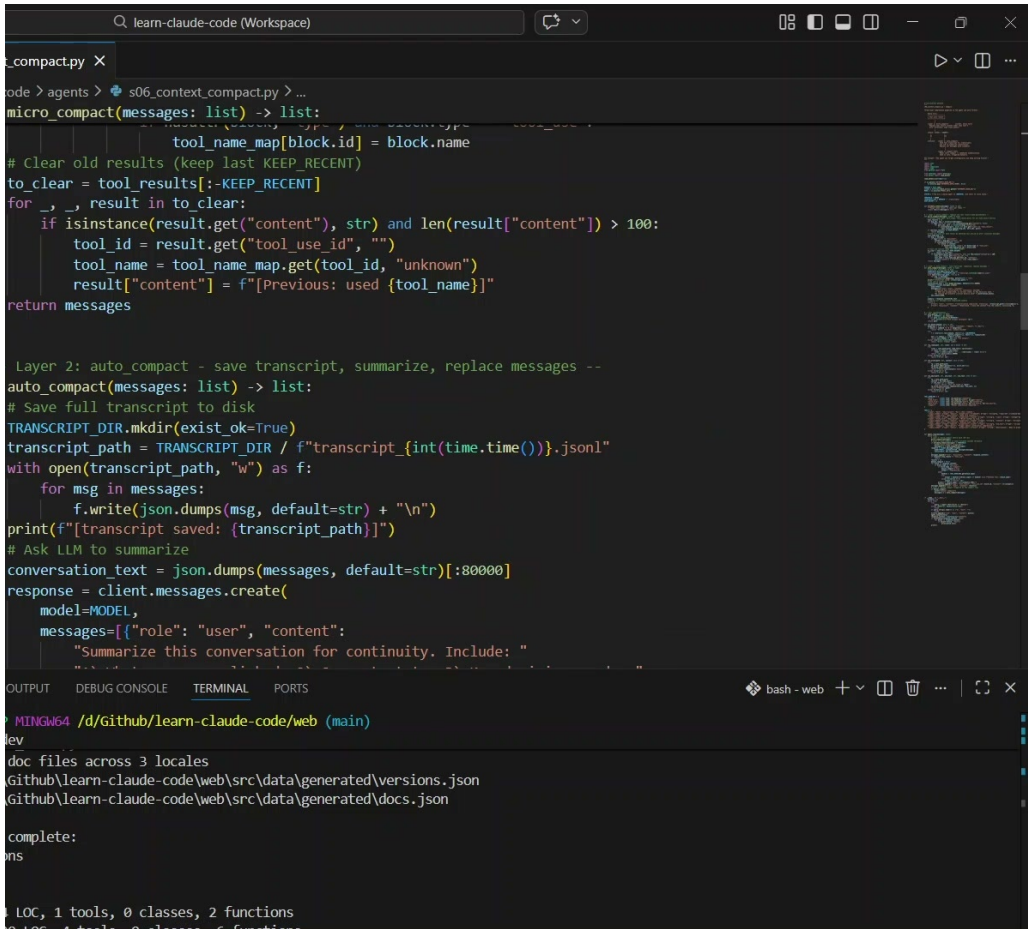


图 1: 在 VS Code 中为 micro_compact、auto_compact 和主循环设置断点。¹

启动调试 (Run and Debug), 选择第一条 Prompt:

```
1 Read every Python file in agents directory one by one
```

Listing 1: 实验用的第一条 Prompt

这条 Prompt 会让模型多次调用工具读取文件,从而产生足够多的工具结果,便于观察 micro_compact 的替换行为。

1.3 第一次进入 micro_compact

当程序首次停在 micro_compact 时, 消息列表 messages 中只有一条用户消息。此时函数内部会尝试收集所有 role == "user" 且 content 为列表类型的消息, 从中提取 tool_result 类型的块。

```
1 for msg in messages:
2     if msg["role"] == "user" and isinstance(msg["content"], list):
3         for block in msg["content"]:
4             if block.get("type") == "tool_result":
5                 tool_results.append({
6                     "msg_idx": msg_idx,
7                     "part_idx": part_idx,
```

¹ 视频画面时间区间: 00:00:25-00:01:20。

```
8         "block": block,
9     })
```

Listing 2: `micro_compact` 中收集旧工具结果的核心逻辑（示意）

由于此时消息列表里只有一条用户消息,且不含任何 `tool_result`,因此收集到的 `tool_results` 为空。接着函数会估算当前消息的 token 数(实现上把消息字符串长度除以 4),得到约 20 多个 token,远低于 5 万的阈值,于是直接返回,不做任何替换。

token 估算的简化公式

源码中 `estimate_tokens` 的实现为 $\text{token} \approx \text{len}(\text{str}(\text{messages}))/4$ 。这是一个快速但粗略的估算:对英文来说,一个 token 平均约对应 4 个字符;对中文则会有所偏差,但足以用于触发阈值判断。

1.4 第二轮与第三轮:开始收集 `tool_results`

随着对话继续,模型开始返回工具调用结果。第二轮进入 `micro_compact` 时,消息列表中已经出现了一条包含 `tool_result` 的消息。此时 `tool_results` 列表的长度为 1,小于保留阈值 `keep_recent = 3`,因此依然不会进行替换,只是记录下来。

到了第三轮, `tool_results` 中已经有两条记录。每条记录除了保存 `tool_result` 块本身,还附加了两个索引:

- `msg_idx`: 该 `tool_result` 所在消息在 `messages` 中的位置 (从 0 开始)。
- `part_idx`: 该 `tool_result` 在消息 `content` 列表中的位置 (从 0 开始)。

例如,若某条记录显示 `msg_idx = 2`, `part_idx = 0`,表示它是第 3 条消息中的第 1 个内容块。

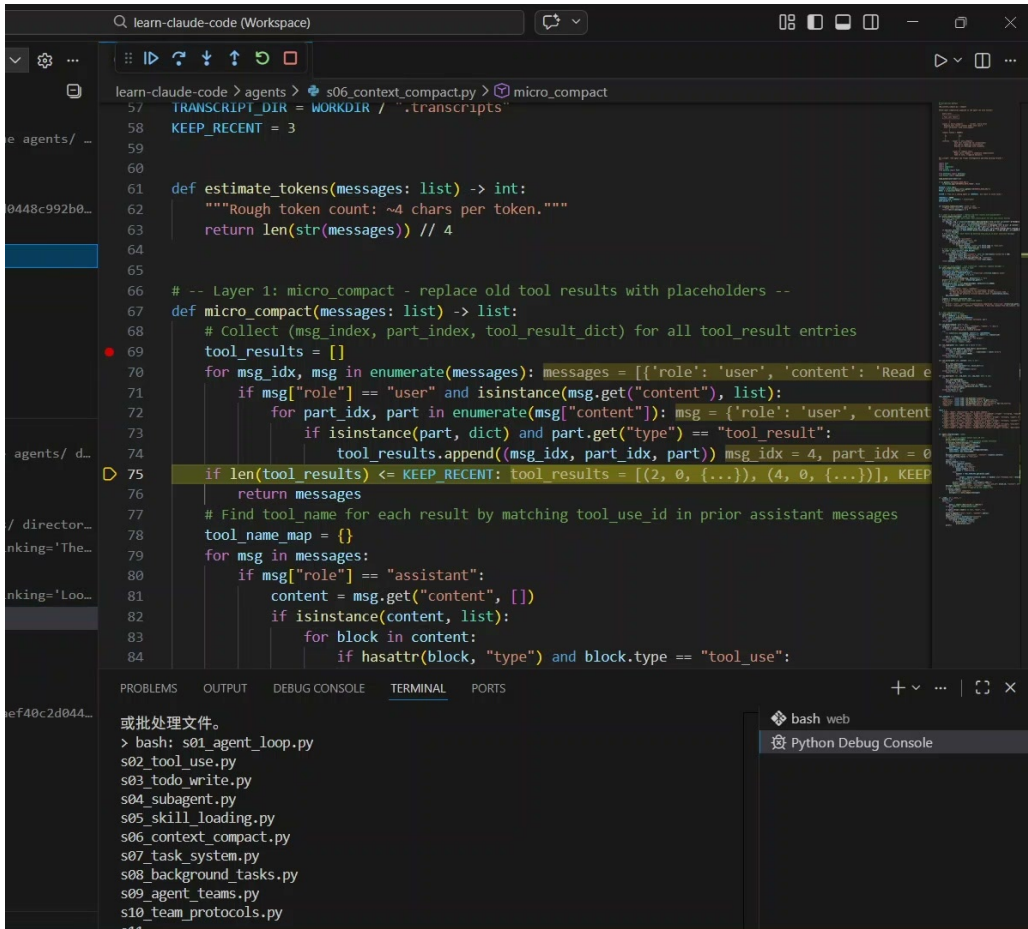


图 2: 调试器中观察 `tool_results` 的 `msg_idx` 与 `part_idx`。²

1.5 首次触发替换：内容被压缩为占位符

当对话进行到后续轮次，`tool_results` 的数量增长到 16 条，超过了 `keep_recent = 3`。此时 `micro_compact` 终于进入了真正的替换分支。

替换前，函数会先遍历所有 `role == "assistant"` 的消息，从它们的 `tool_use` 块中提取 `tool_use_id` 到 `tool_name` 的映射。这是因为即使是同一个工具（如 `bash`），每次调用的 `id` 都不同，必须建立映射才能知道某个 `tool_result` 对应的是哪个工具。

随后，函数对 `tool_results` 中除最近 3 条以外的所有旧结果执行清理：

```
1 for tr in tool_results[:-keep_recent]:
2     content = tr["block"].get("content", "")
3     if isinstance(content, str) and len(content) > 100:
4         tool_id = tr["block"]["tool_use_id"]
5         tool_name = tool_use_map.get(tool_id, "unknown")
6         tr["block"]["content"] = f"previous_used_{tool_name}"
```

Listing 3: 替换旧工具结果为占位符

在调试器中可以清楚地看到：原本长度数百字符的 `tool_result` 内容，被替换成了类似 `previous used bash` 的短字符串，长度骤降到十几个字符。这意味着在后续发给 LLM 的消息列表中，旧的

² 视频画面时间区间：00:04:45–00:06:15。

工具结果几乎不再占用上下文空间。

压缩的副作用

虽然占位符大幅节省了 token，但它也丢弃了旧工具结果的具体内容。如果后续对话需要引用这些旧结果中的细节，模型将无法看到完整信息。因此源码保留了最近 3 条 `tool_result` 不被替换，作为折中。

1.6 本章小结

本章通过 VS Code 调试器，逐步跟踪了 Claude Code 中第一层上下文压缩——`micro_compact` 的执行过程：

- **动机**：随着工具调用次数增加，消息列表中的 `tool_result` 会迅速膨胀，必须加以压缩。
- **机制**：`micro_compact` 在每次 Agent 循环开始时运行，收集所有旧的 `tool_result`，并将其长度超过 100 字符的内容替换为 `previous used <tool_name>` 占位符。
- **保留策略**：最近 3 条 `tool_result` 不被替换，以免丢失刚刚产生的关键信息。
- **token 估算**：源码用 长度/4 快速估算当前消息是否接近 5 万 token 阈值，为下一章的 `auto_compact` 做铺垫。

下一章将进一步探讨：当微观压缩不足以控制上下文长度时，`auto_compact` 如何把整段对话摘要成两条消息，以及其中隐藏的一个源码 bug。

2 `micro_compact` 的首次执行与工具结果压缩机制

2.1 为什么要先进行微小压缩？

在 Claude Code 的主循环 (agent loop) 中，每一轮对话开始前都会先调用 `micro_compact`。为什么要这样做？

想象一下：LLM 每轮都会调用多个工具 (`read`、`edit`、`bash` 等)，这些工具的返回结果可能非常长——一个文件的内容动辄几千字符。如果不加处理，这些历史结果会迅速堆满上下文窗口。但另一方面，我们又不能把所有旧结果都删掉，因为 LLM 有时需要参考之前读取过的文件内容。

`micro_compact` 的设计目标就是在“保留信息”和“控制长度”之间做一个轻量级的平衡：它不会删除消息，而是把过长的工具结果替换为一条极短的提示语，让 LLM 知道“这个工具之前用过”，从而把上下文长度压下来。

`micro_compact` 是三层压缩中的第一层，特点是：

- 只在客户端本地执行，不额外请求 LLM；
- 只替换长度超过阈值的历史工具结果；
- 保留最近几条结果不做处理，避免影响当前轮次的推理。

2.2 tool_results 的收集与观察

当我们通过 VS Code 调试进入 `micro_compact` 时，可以在 Watch 面板中观察到一个名为 `tool_results` 的列表在逐步增长。

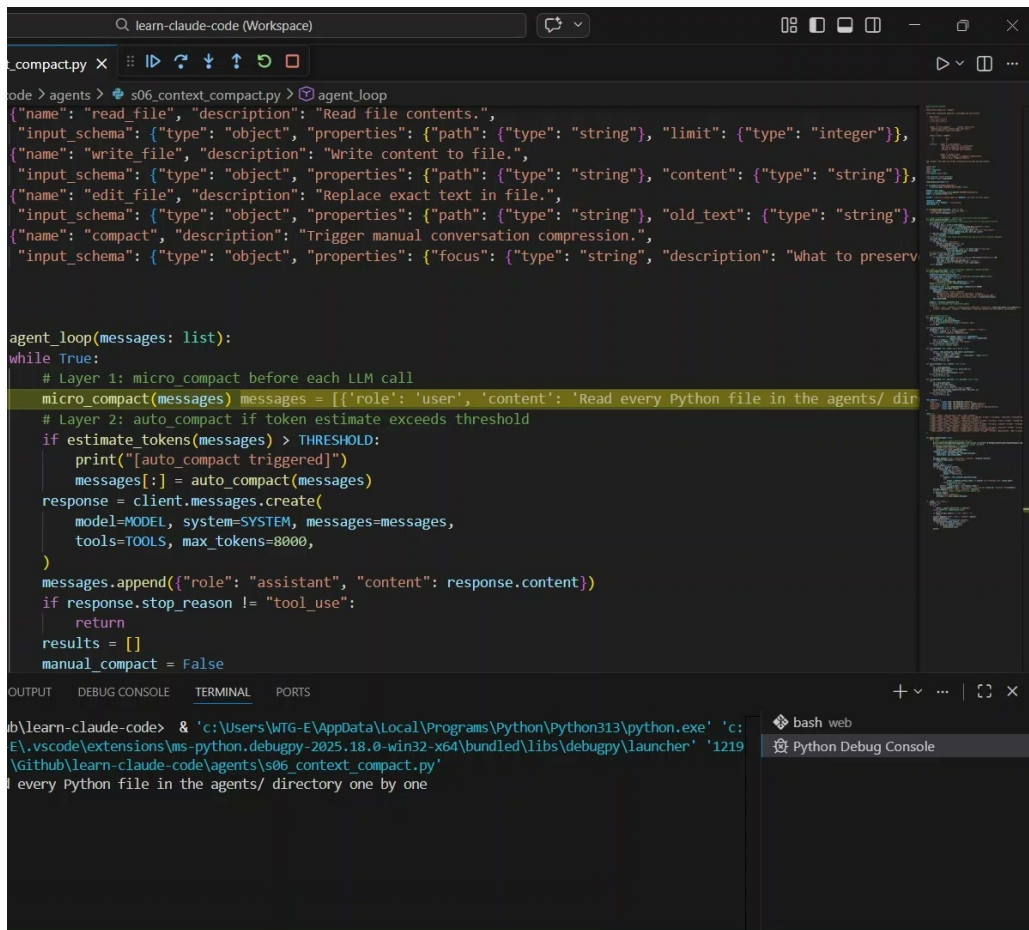


图 3: 在 VS Code Watch 面板中观察 `tool_results` 从 1 条增长到 16 条的过程。³

`tool_results` 的收集逻辑大致如下：函数遍历当前 `messages`，找出所有 `role == user` 且 `content` 为列表类型的消息，然后将其中的 `tool_result` 块提取出来。每一个被提取的元素都会带上两个索引：

- `msg_idx`: 该 `tool_result` 在 `messages` 中的位置；
- `part_idx`: 该 `tool_result` 在对应消息 `content` 列表中的位置。

例如，调试中观察到的 (2, 0) 和 (4, 0) 表示：第 2 条消息和第 4 条消息中的第一个 `tool_result` 块。这种双索引设计是为了在后续替换时能够精确定位回原始消息。

2.3 构建 `name_map`: 工具 ID 到工具名称的映射

当 `tool_results` 的数量超过 `keep_recent` (默认值为 3) 时，`micro_compact` 就会开始对“较老”的结果进行压缩处理。但在替换之前，它需要先解决一个问题：如何把工具结果和对应的工具名称关联起来？

³ 视频画面时间区间：00:02:00–00:07:15。

答案藏在 LLM 返回的 assistant 消息中。micro_compact 会遍历所有 role == assistant 的消息，查找其中 type == tool_use 的块，并构建一个字典 name_map：

```
1 name_map = {}
2 for msg in messages:
3     if msg["role"] == "assistant" and isinstance(msg["content"], list):
4         for block in msg["content"]:
5             if block.get("type") == "tool_use":
6                 name_map[block["id"]] = block["name"]
```

Listing 4: 构建 name_map 的核心逻辑示意

即

使是同一个工具（例如 bash），每次调用时 tool_use 的 id 也是不同的。因此不能简单地按工具名去匹配，而必须通过唯一的 id 来建立一一映射。

2.4 替换规则：长度大于 100 即压缩

构建好 name_map 后，micro_compact 取出除最近 3 条以外的所有 tool_results（即 to_clear），对它们逐一检查：

1. 该 tool_result 的 content 是否为字符串类型；
2. 该字符串长度是否大于 100。

如果两个条件都满足，就把原来的 content 替换为：

```
previous used <tool_name>
```

其中 <tool_name> 就是通过 name_map 查到的工具名称。

```
1 for item in to_clear:
2     content = item["content"]
3     if isinstance(content, str) and len(content) > 100:
4         tool_id = item["tool_use_id"]
5         tool_name = name_map.get(tool_id, "unknown")
6         item["content"] = f"previous_used_{tool_name}"
```

Listing 5: micro_compact 的替换逻辑核心示意

在调试过程中可以清晰地看到：原本一个 bash 工具返回的长文本（长度远大于 100），在 F10 单步执行后瞬间变成了 previous used bash——从几百字符压缩到十几个字符。

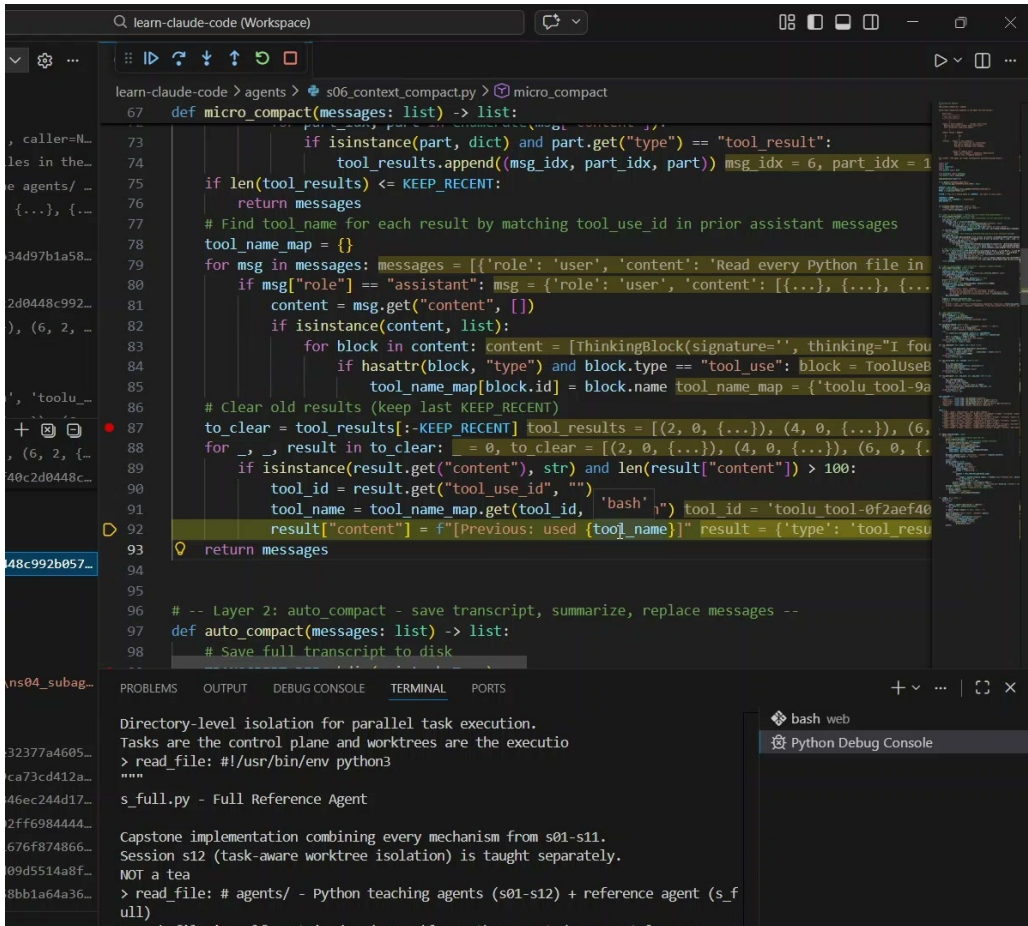


图 4: Watch 面板中某条 tool_result 的 content 被替换为 previous used bash。⁴

2.5 第一次压缩的实际效果

完成替换后，如果我们展开 messages 查看，会发现历史消息中的 tool_result 块已经大幅瘦身。这意味着：

- 下一轮发送给 LLM 的上下文总长度显著降低；
- LLM 仍然能从“previous used bash”这样的提示中推断出“之前执行过某个 bash 命令”；
- 最近 3 条工具结果保持原样，确保当前轮次所需的详细信息不会丢失。

这

种压缩是有损的：如果后续对话需要引用某条旧工具结果中的具体数据（例如某次 read 返回的特定行号），而那条结果已经被替换为“previous used read”，LLM 就无法再看到原始内容了。因此 micro_compact 只处理“较老”的结果，并在长度阈值（100）之上才触发，以尽量减少信息损失。

2.6 本章小结

本章通过 VS Code 调试，追踪了 micro_compact 的完整执行流程：

⁴视频画面时间区间：00:10:00-00:13:30。

1. **收集**: 从 `messages` 中提取所有 `tool_result`, 形成 `tool_results` 列表;
2. **映射**: 遍历 `assistant` 消息中的 `tool_use` 块, 构建 `id -> name` 的 `name_map`;
3. **筛选**: 排除最近 3 条结果, 对其余结果检查 `content` 长度;
4. **替换**: 凡长度超过 100 的字符串内容, 一律替换为 `previous used <tool_name>`。

这一机制是 Claude Code ”三层上下文压缩” 中的第一层, 它用极低的计算成本实现了对历史工具结果的有损压缩, 为后续可能触发的 `auto_compact` 争取了更大的上下文余量。

3 上下文压缩: `auto_compact` 的触发与入口条件

3.1 动机: 为什么需要上下文压缩?

在与大语言模型 (LLM) 进行长会话时, 上下文窗口是有限的资源。随着对话轮次增加, `messages` 列表中会积累大量的用户输入、模型回复以及工具调用结果。如果不加以控制, 最终会导致 token 消耗超出模型承受范围, 甚至触发长度限制错误。

Claude Code 采用了一套分层的上下文压缩策略。本章聚焦于其中一层——`auto_compact`: 当客户端检测到上下文规模可能过大时, 主动触发压缩, 将历史消息替换为更精简的摘要形式。理解这一机制的触发条件、token 估算逻辑以及模型主导的 `compact` 入口, 是掌握 Claude Code 长会话能力的关键。

上

下文压缩的本质是用信息密度换窗口空间。压缩得越激进, 节省的 token 越多, 但丢失细节的风险也越大。

3.2 `auto_compact` 的入口与触发条件

在 Agent 的主循环中, `auto_compact` 的调用发生在两个位置:

- 第 201 行: 在常规流程中, 根据 token 估算结果决定是否压缩。
- 第 228 行: 作为另一条分支的兜底压缩逻辑。

为了让调试能够直接进入 `auto_compact`, 视频中使用了一个特殊的 prompt: `Use a compact tool to manually compress the conversation`。这个 prompt 会引导模型在 `tool_use` 响应中返回 `compact` 工具调用, 从而将 `manual_compact` 标志设为 `True`, 最终触发 `auto_compact` 的执行。

```
1 # 当 tool_name 为 "compact" 时, 不调用具体 handler
2 # 而是设置 manual_compact = True
3 manual_compact = True
```

Listing 6: 手动触发 `compact` 后的标志位设置

3.3 Token 估算: estimate_tokens 的逻辑

在决定是否进入压缩之前, Claude Code 会先调用 `estimate_tokens(messages)` 来估算当前上下文的 token 消耗。这个估算函数的实现非常简洁:

```
1 def estimate_tokens(messages):
2     return len(str(messages)) // 4
```

Listing 7: estimate_tokens 的简化实现

也就是说, 它把整个 `messages` 列表转成字符串, 然后按每 4 个字符约等于 1 个 token 的比例进行估算。

Token 估算的直观理解

OpenAI 的 Tokenizer 工具可以帮助我们建立直觉:

- "hello" (5 个字符) 1 token
- "嗨" (2 个字符) 1 token

可见, 不同语言的字符密度不同, 4 字符/token 只是一个工程上的近似值, 用于快速判断是否需要压缩, 而非精确计费。

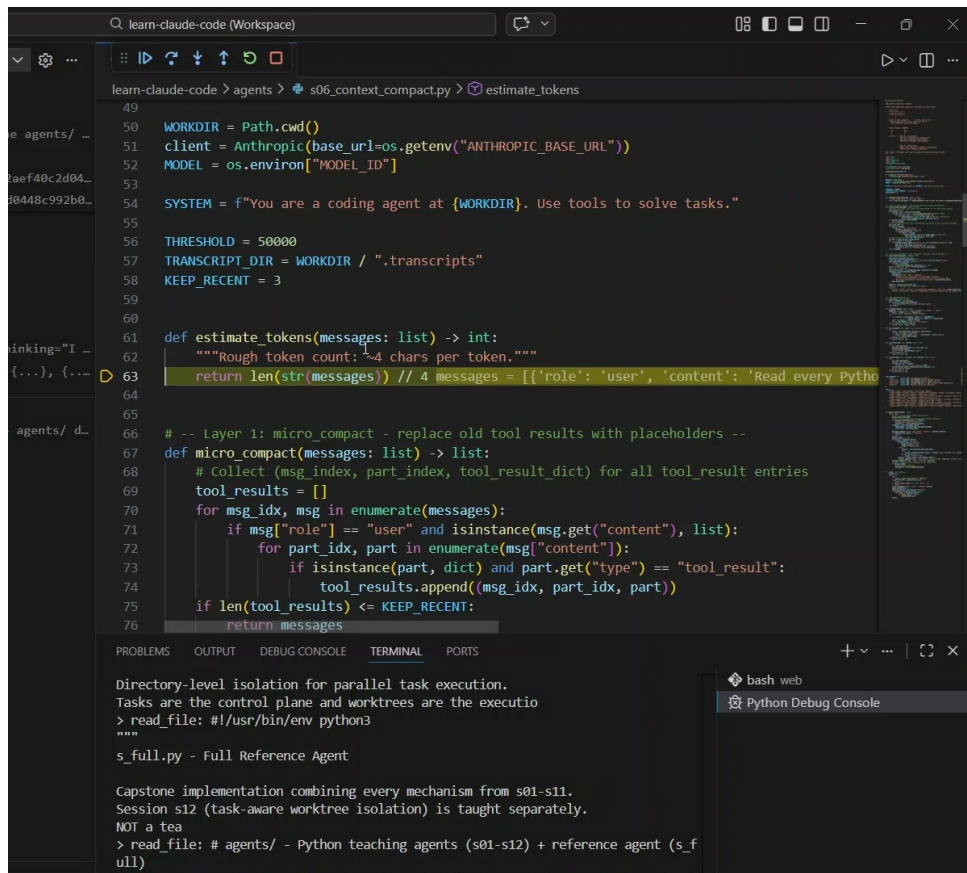


图 5: Token 估算逻辑: 字符串长度除以 4 与 50,000 阈值比较⁵

⁵ 视频画面时间区间: 00:02:47-00:03:21。

3.4 50,000 Token 阈值：压缩的临界点

估算出的 token 数会与一个固定阈值进行比较：

```
1 THRESHOLD = 50000
2 if estimate_tokens(messages) > THRESHOLD:
3     # 触发 auto_compact
```

Listing 8: 阈值判断逻辑

这个 50,000 的阈值是客户端预设的警戒线。在视频中的实验场景下,由于对话轮次较少, `estimate_tokens` 返回的值只有 20 左右,远未达到阈值。因此,常规的自动触发路径很难在短会话中命中。这也解释了为什么视频中需要通过特殊 prompt 来**手动诱导**进入 `auto_compact`,以便观察其内部行为。

5

0,000 阈值是一个**客户端硬编码**的保守值。实际生产环境中,如果模型上下文窗口更大或更小,这个阈值可能需要调整,但源码中并未暴露为可配置参数。

3.5 模型主导的 compact：让 LLM 自己决定何时压缩

除了客户端基于 token 阈值自动触发, Claude Code 还向模型暴露了一个名为 `compact` 的特殊工具。与普通工具不同, `compact` 没有对应的业务逻辑 handler——它 purely 是一个元控制 (meta-control) 标记。

当模型感知到对话历史已经过于冗长、需要摘要时,它可以主动在 `tool_use` 响应中返回 `compact` 调用。Agent Loop 检测到这个标记后,会将 `manual_compact` 设为 `True`,随后同样进入 `auto_compact` 分支执行摘要压缩。

关键洞察：第二层（客户端自动触发）和第三层（模型主动调用）最终都落入同一个 `auto_compact` 实现。这是一种“双入口、单出口”的设计：客户端的粗糙估算提供工程兜底,而模型的自我感知则赋予系统额外的灵活性。

在视频中,讲师正是通过下面这条明确的 prompt,成功诱导模型发出了 `compact` 调用：

```
1 Use a compact tool to manually compress the conversation
```

Listing 9: 诱导模型调用 compact 工具的 Prompt

这一机制让 LLM 获得了主动调控上下文长度的能力——它不再被动等待客户端规则触发,而是可以基于自身对注意力负载的“元认知”提前请求压缩。

3.6 本章小结

本章聚焦于 `auto_compact` 的触发路径与入口条件：

- **双入口设计**: `auto_compact` 在 Agent 循环的两个位置被调用,常规路径依赖于 `estimate_tokens(messages) > 50000` 的阈值判断;模型也可通过 `compact` 工具主动触发。
- **Token 估算**: 采用简洁的启发式算法——字符串总长度除以 4,作为是否需要压缩的快速判断依据。

- **手动诱导**：在短会话中难以达到 50k 阈值时，可通过特殊 prompt 诱导模型发出 `compact` 工具调用，将 `manual_compact` 设为 `True`。
- **元工具机制**：`compact` 是一个没有具体 handler 的元控制工具，它让 LLM 获得了自我调控上下文长度的能力。

理

解 `auto_compact` 的关键在于把握其**分层取舍**的设计哲学：用客户端的保守估算做兜底，同时保留模型智能的灵活性，在信息完整性和上下文长度之间取得平衡。

4 Auto Compact：磁盘持久化与 LLM 摘要压缩

4.1 动机：为什么要做“自动压缩”

在上一节中，我们了解了 `micro_compact` 如何在内存中把旧工具结果替换成简短的占位符。然而，这种替换只是“微压缩”——当对话轮数持续增长，整个消息列表的 token 消耗终究会逼近甚至超过模型上下文窗口的上限。此时，仅靠替换几个工具结果已经杯水车薪，必须有一种更激进的压缩手段：把整条对话“浓缩”成一份摘要。

核心目标：在 token 估算值超过阈值（50k）或模型主动调用 `compact` 工具时，将当前所有消息序列化到磁盘，并调用 LLM 生成摘要，最后用摘要替换掉原始消息，从而把百条消息压缩成两条。

4.2 创建 `.transcripts` 目录并保存原始消息

当进入 `auto_compact` 函数后，代码做的第一件事不是直接调用模型，而是先把当前内存中的完整对话“存档”到本地磁盘。这一步既是调试利器，也是数据安全兜底：万一压缩过程出错，开发者仍然可以从 JSON 文件中回溯完整上下文。

```

1 transcript_dir = Path.cwd() / ".transcripts"
2 transcript_dir.mkdir(parents=True, exist_ok=True)
3
4 transcript_path = transcript_dir / f"{datetime.now().isoformat()}.json"
5 transcript_path.write_text(
6     json.dumps(messages, indent=2, ensure_ascii=False),
7     encoding="utf-8",
8 )
9 print(f"Transcript saved to {transcript_path}")

```

Listing 10: 创建 transcripts 目录并序列化消息

如代码所示，`transcript_dir` 被定义为当前工作目录下的 `.transcripts` 文件夹。随后，代码使用 Python 标准库 `json.dumps` 将整个 `messages` 列表以缩进格式写入一个以当前时间命名的 `.json` 文件中。

占位图：VS Code 文件浏览器中显示刚生成的
.`transcripts/2024-01-01T12:00:00.json`

图 6: 序列化后的对话文件被保存到 `.transcripts` 目录下⁶

`json.dumps` 的作用

`json.dumps` 把 Python 对象（这里是字典/列表）转换成 JSON 字符串。参数 `indent=2` 让输出带有缩进，便于人类阅读；`ensure_ascii=False` 则保证中文字符不会被转义成 `\uXXXX`，从而直接以可读形式写入文件。

4.3 LLM 摘要调用：summarize this conversation for continuity

存档完成后，代码开始向 LLM 发起一次额外的 API 调用，目的是让模型为当前对话生成一份“连续性摘要”。提示词结构如下：

```
1 conversation_text = str(messages)[:80_000]
2
3 response = client.messages.create(
4     model=COMPACTION_MODEL,
5     max_tokens=4096,
6     messages=[{
7         "role": "user",
8         "content": (
9             "Summarize this conversation for continuity."
10            "Include all important decisions, facts, and context.\n\n"
11            f"{conversation_text}"
12        ),
13    }],
14 )
```

Listing 11: 构造摘要请求

这里有几个关键点值得注意：

1. **模型选择**：摘要调用使用的是专门的 `COMPACTION_MODEL`，通常是一个轻量或快速的模型，以降低成本和延迟。
2. **提示词意图**：`summarize this conversation for continuity` 明确要求摘要必须保留“连续性”——不是泛泛的总结，而是要让模型在后续轮次中仍能基于这份摘要继续工作。
3. **角色固定为 user**：即使原始消息里有 `system`、`assistant`、`user` 多种角色，这里统一把它们打包成一条 `role=user` 的消息发送给模型。

4.4 80,000 字符截断：被丢弃的尾部信息

在上述代码中，最引人注目的细节是这行：

```
1 conversation_text = str(messages)[:80_000]
```

⁶视频画面时间区间：00:18:10–00:19:30。

代码并没有把整个消息列表全部喂给摘要模型，而是只取了前 80,000 个字符。这意味着：如果对话历史非常长，**80,000 字符之后的内容会被直接丢弃**，不会进入摘要过程。

潜在风险：截断策略是“取前 80,000 字符”，而不是“取最近 80,000 字符”。因此，如果用户在对话后期给出了关键约束（例如“不要修改某文件”或“改用另一种方案”），而这些信息恰好位于 80,000 字符之后，那么摘要将完全丢失这些指令，导致后续轮次模型行为偏离用户预期。

占位图：调试器中显示 `conversation_text` 的长度为 80000

图 7: 调试器观察到 `conversation_text` 被截断至 80,000 字符⁷

4.5 从模型响应到压缩后的消息列表

LLM 返回的摘要内容会从响应对象中提取出来，随后替换掉原始消息列表：

```
1 summary = response.content[0].text
2
3 messages = [
4     {"role": "user", "content": f"Previous conversation summary:\n{summary}"},
5     {"role": "assistant", "content": "Understood. I'll continue based on that summary."},
6 ]
```

Listing 12: 用摘要替换原始消息

无论之前有多少条消息——十条、五十条、一百条——经过 `auto_compact` 之后，都会被压缩成上述 **两条消息**：

- 第一条：user 角色，携带完整的摘要文本；
- 第二条：assistant 角色，一个形式上的确认回复。

这种压缩力度是 `micro_compact` 无法比拟的。如果说 `micro_compact` 是在修剪枝叶，`auto_compact` 就是直接把整棵树替换为一份“树的说明书”。

压缩效果：`auto_compact` 将任意长度的历史消息列表替换为固定两条消息，从而把上下文 token 数量从可能的几万瞬间降到几百，为后续对话腾出充足空间。

4.6 本章小结

本章聚焦于 Claude Code 的 `auto_compact` 机制，揭示了它在激进压缩上下文时所采用的完整流程：

⁷ 视频画面时间区间：00:19:50–00:20:50。

- **磁盘存档**: 首先在当前目录下创建 `.transcripts` 文件夹, 并使用 `json.dumps` 把完整消息序列化到时间戳命名的 JSON 文件中, 便于调试与回溯。
- **LLM 摘要**: 向 `COMPACTION_MODEL` 发送提示词 `summarize this conversation for continuity`, 请求生成一份保留关键决策与上下文的连续性摘要。
- **80k 截断**: 在构造摘要输入时, 只截取 `str(messages)` 的前 80,000 个字符, 尾部信息被直接丢弃——这是保证摘要请求可控的代价, 也是潜在的信息丢失风险点。
- **消息替换**: 最终将原始所有消息替换为两条新消息 (摘要 + 确认), 实现从“百条消息”到“两条消息”的跨越式压缩。

理解 `auto_compact` 的运作方式, 有助于我们在使用 Claude Code 时意识到: 当对话变得非常长时, 某些后期指令可能会因为 80k 截断而“消失”在摘要中。在关键约束出现时, 及时开启新会话或手动整理上下文, 仍然是保障模型行为可控的最佳实践。

5 调试中的意外发现: 三层上下文压缩与潜藏的 Bug

在前几节中, 我们已经跟随源码走过了 Agent Loop 的主流程、工具调用与结果回传。本节将聚焦于一个更为精巧的机制——**上下文压缩 (Context Compaction)**。通过单步调试, 我们不仅会还原 Claude Code 宣称的「三层压缩」框架, 还会在源码中撞见一个真实的 Bug。

5.1 从 `micro_compact` 开始: 旧工具结果的「微瘦身」

在 `agent.py` 的主循环中, 每次进入迭代前都会先调用 `micro_compact(messages)`。它的职责是对历史消息做一次轻量级的「瘦身」: 把过长的旧 `tool_result` 内容替换为一句占位符。

```

1 # 1. 收集所有 user 消息中的 tool_result (保留最近 3 条)
2 # 2. 对更早的 tool_result, 若 content 长度 > 100, 则替换为
3 #    "previous used <tool_name>"
4 # 3. 通过 tool\_id -> tool\_name 的映射, 确保占位符有意义

```

Listing 13: `micro_compact` 的核心逻辑片段

调试过程中可以清晰地看到这一过程:

- 当消息中只有 12 条 `tool_result` 时, 由于 `keep_recent = 3`, `micro_compact` 什么都不做, 直接返回原消息列表。
- 随着对话轮次增加, `tool_results` 累积到 16 条时, 系统开始清理前 13 条。原本可能长达数百字符的 `bash` 输出, 被瞬间压缩成 `previous used bash` 这样短短十几个字符。

直觉解释: `micro_compact` 就像一位速记员, 它不会删除整条记录, 只是把「旧账本的详细流水」折叠成一行摘要, 让 LLM 知道「这里曾经调用过某个工具」即可。

5.2 `auto_compact`: 客户端的 5 万 Token 警戒线

`micro_compact` 之后, 代码会估算当前消息列表的 token 消耗。估算方式非常朴素:

```

1 def estimate_tokens(messages):
2     return len(str(messages)) // 4

```

Listing 14: Token 估算逻辑

这里用字符串总长度除以 4 来近似 token 数——对于英文单词而言，这大致相当于把平均每个 token 视为 4 个字符。

估算公式背后的考量

OpenAI 的 tokenizer 中，常见英文单词（如 hello）15 个字符可能只对应 1 个 token；而中文或特殊符号的压缩率则不同。源码采用「除以 4」是一种快速、保守的工程近似，目的是在客户端本地就能判断是否需要触发压缩，而不必每次都调用真实的 tokenizer API。

当估算值超过 50,000 这个阈值时，auto_compact 就会被自动触发。它的行为非常激进：

1. 把当前工作目录下的所有消息序列化保存到 .transcripts/ 目录；
2. 向 LLM 发送一条摘要请求：summarize this conversation for continuity；
3. 用返回的 summary 替换掉原有的全部消息，最终只剩下两条：系统提示 + 摘要。

占位图：三层上下文压缩框架示意图（第一层微观替换、第二层客户端阈值触发、第三层模型主动调用）

图 8: 三层上下文压缩框架示意图。第一层在工具结果层面做微观替换；第二层由客户端基于 50k token 阈值主动触发摘要；第三层则由模型自身判断是否需要调用 compact 工具。⁸

5.3 模型主导的第三层压缩：手动触发 compact

除了客户端自动触发，Claude Code 还向模型暴露了一个名为 compact 的工具。当模型认为对话历史已经过于冗长、需要摘要时，它可以主动调用这个工具。

在调试中，讲师通过一条明确的 prompt——use a compact tool to manually compress the conversation——诱导模型发出了 compact 调用。与普通的 tool_use 不同，compact 并没有对应的 tool_handler 函数，它只是一个字符串标记。当 Agent Loop 检测到这个标记时，会把 manual_compact 标志设为 True，随后同样进入 auto_compact 的逻辑。

无

论是客户端自动触发（auto_compact）还是模型主动调用（compact），最终都落入同一个压缩函数。这意味着「三层压缩」中的第二层和第三层，本质上共享的是同一套实现，只是触发条件不同。

5.4 源码中的 Bug：thinking block 没有 text 属性

在调试 auto_compact 时，一个意外的错误跳了出来。源码第 115 行试图这样提取摘要内容：

⁸视频画面时间区间：00:22:45–00:23:45。

```
1 summary = response.content[0].text
```

Listing 15: 存在问题的代码（示意）

然而，当模型返回的 `content[0]` 是一个 `thinking block` 而非 `text block` 时，这个对象并没有 `.text` 属性，程序直接抛出了 `AttributeError`。

Bug 分析：这段代码假设了 LLM 返回的第一个 `content block` 永远是 `text`，但在实际运行中，模型可能先输出一个 `thinking block`。这是一个典型的「类型假设过强」导致的边界错误。修复方式应当是遍历 `response.content`，找到第一个类型为 `text` 的 `block` 再提取其 `text` 字段。

这个 Bug 也提醒我们：即使是 Anthropic 官方出品的工具，在对接 LLM 的流式/多模态响应时，也需要对 `content block` 的类型做防御性处理。

5.5 80k 字符截断：压缩带来的隐性风险

在 `auto_compact` 内部，还有一处值得警惕的硬编码：

```
1 conversation_text = str(messages)[:80000]
```

Listing 16: conversation text 的截断逻辑

系统在向 LLM 请求摘要之前，只保留消息列表序列化后的前 8 万个字符，超出的部分被直接丢弃。

风险警示：如果用户在对话后期给出了关键约束——例如「不要修改某个文件」或「必须使用某个特定版本」——而这些信息恰好落在 80k 字符之后，那么摘要请求将完全看不到这些约束。压缩后的 `summary` 会丢失这些重要上下文，导致模型在后续轮次中「失忆」并做出违背用户意图的操作。

这揭示了上下文压缩的一个根本张力：**无限会话的代价是信息损耗**。工程师必须在「会话长度」与「信息保真」之间做权衡，而 80k 的硬截断显然是一种偏向工程简洁性的取舍。

5.6 本章小结

本章通过源码调试，还原了 Claude Code 的三层上下文压缩机制：

1. **micro_compact**: 对历史 `tool_result` 做微观替换, 把长输出折叠为 `previous used <tool_name>`;
2. **auto_compact**: 客户端基于 50k token 估算阈值, 自动将整条对话摘要为两条消息;
3. **模型主动 compact**: LLM 自行判断对话是否过于冗长, 并通过 `compact` 工具触发同样的摘要逻辑。

同时，我们也发现了两个值得注意的问题：

- 源码在处理 LLM 响应时存在类型假设错误（`thinking block` 无 `text` 属性）；

- 80k 字符的硬截断可能导致后期关键约束被丢弃，引发模型行为偏离用户预期。

理解这些细节，不仅有助于我们正确使用 Claude Code，也为设计自己的长上下文 Agent 提供了宝贵的工程参考。

6 总结与延伸

6.1 讲师的收官之言

在视频的尾声，讲师用一段简洁有力的总结收束了整堂课：

「正是因为大模型的上下文窗口是有边界的，所以我们要想办法去压缩。但压缩其实有时候也会带来问题——在这段代码里面，它只取了前 8 万个字符，把后面的内容舍弃掉了。这就意味着，如果你在对话后期提到了什么关键信息，比如让他不要做什么，那他只提取了前 8 万个字符，就把你的那条约束忽略掉了。大模型后续就不知道你要做什么，可能做的就不满足你的要求。这一点我们要注意。」

这段话点出了整个上下文压缩设计的核心张力：**边界与保真不可兼得**。工程师用算法换来了「无限会话」的幻觉，但幻觉之下，信息的损耗是真实存在的。

6.2 核心论断的结构化提炼

纵观整节课，我们可以把讲师的源码解读提炼为三条相互嵌套的核心论断：

1. **压缩是分层的**。Claude Code 没有依赖单一策略，而是构建了三层由轻到重的防御体系：微观替换（micro）、客户端阈值触发（auto）、模型自主决策（model-decided）。这种分层设计让压缩成本与触发频率成反比——越频繁发生的场景，压缩代价越小。
2. **触发条件是多元的**。同一套 `auto_compact` 实现，可以由两种完全不同的条件触发：一种是客户端基于字符串长度除以 4 的粗糙估算，另一种是 LLM 基于对自身注意力负载的「元认知」。这种「双入口、单出口」的设计，既保证了工程上的兜底，又保留了模型智能的灵活性。
3. **压缩是有损的**。80k 字符的硬截断、50k token 的估算阈值、以及 summary 对原始对话的不可逆替换，都意味着关键信息可能在压缩过程中被静默丢弃。这不是实现缺陷，而是架构层面的固有取舍。

6.3 三层机制的交叉关联

如果把三层压缩比作一座水坝的泄洪系统，它们的关系可以这样理解：

- **micro_compact** 是日常的「渗流孔」，在水位（token 数）还不高时，先把旧工具结果这些「泥沙」过滤掉，减缓水位上涨速度。
- **auto_compact** 是自动启用的「溢洪道」，当水位逼近 50k 警戒线时，一次性倾泻掉大量历史对话，只保留最核心的摘要。
- **模型主导的 compact** 则是由大坝自身传感器判断后手动开启的「应急闸门」——它基于对当前水情（对话复杂度）的感知，决定是否提前泄洪。

三层之间并非简单的递进关系，而是**协同过滤**：micro 层减少了进入 auto 层的频率；auto 层的存在又让模型不必频繁动用自身的判断能力；而模型的判断能力，又弥补了客户端估算（长度除以 4）可能带来的误判。

6.4 工程实践的启示

从这段源码中，我们可以提取出若干对实际开发有直接指导意义的 takeaways：

给 Agent 开发者的五条建议

1. **先做轻量压缩，再做重量级摘要**。不要等 token 满了才想起压缩，像 `micro_compact` 这种「微瘦身」成本极低，应该作为默认动作。
2. **本地估算可以粗糙，但阈值要留足余量**。用 `len(str(messages)) // 4` 代替真实 tokenizer 是一种合理的工程权衡，但 50k 的阈值需要为估算误差留出安全边界。
3. **给模型一个「自我调控」的出口**。暴露 `compact` 这类元工具，让 LLM 能在感知到上下文过载时主动干预，比纯粹依赖客户端规则更灵活。
4. **对 LLM 响应做防御性解析**。`thinking block` 的 Bug 提醒我们，`content block` 的类型不可假设，遍历 + 类型检查是更稳健的做法。
5. **显式告知用户压缩的代价**。80k 截断意味着后期约束可能丢失，如果产品面向终端用户，应当在 UI 层给出提示，或在摘要中保留用户最近的几条关键指令。

6.5 开放问题与未来方向

这节课的源码解读也留下了一些值得继续深挖的问题：

1. **摘要质量如何量化**？当前实现直接把 LLM 生成的 `summary` 替换掉全部历史，但没有对摘要的完整性或准确性做任何校验。是否存在一种「摘要-回滚」机制，让模型在发现摘要遗漏关键信息时能够恢复部分原始对话？
2. **截断策略能否更智能**？80k 的硬截断是一种「先进先出」的粗暴策略。如果改为基于语义重要度的选择性保留（例如保留所有 `system` 消息、用户否定性约束、以及最近的 N 轮对话），是否能在同等压缩率下获得更高的保真度？
3. **三层压缩的触发频率分布如何**？在真实使用场景中，`micro`、`auto`、`model-decided` 各自被触发的比例是多少？是否存在 `auto` 层被过度触发，导致用户体验出现「断崖式失忆」的情况？
4. **token 估算能否更精确**？随着多语言、代码、符号混合的对话越来越常见，「长度除以 4」的误差可能显著放大。引入轻量级的本地 tokenizer（如 `tiktoken`）是否值得？

6.6 最后的延伸思考

Claude Code 的上下文压缩设计，本质上是在回答一个更宏大的问题：**当记忆空间有限时，一个智能体应当如何遗忘**？

人类的大脑通过睡眠中的记忆巩固来解决这个问题——重要的细节被强化，琐碎的感知被淡化。Claude Code 的「三层压缩」则是一种工程化的、基于规则的「机器遗忘」：它用 `previous used bash` 淡化旧工具输出，用 `summary` 重构长篇对话的叙事骨架，用 80k 截断强行丢弃溢出的记忆。

这种设计并不完美，但它揭示了一个关键方向：**未来的长上下文 Agent，竞争的核心将不再是「能记住多少」，而是「知道该遗忘什么」**。在这个意义上，Claude Code 的 compact 机制，正是通往更高级记忆管理的一块重要基石。